

Relatório Técnico — Titanic: Machine Learning from Disaster

Evolução dos Modelos Preditivos em 4 Iterações

Projeto: Desafio 03 — Titanic: Machine Learning from Disaster (Kaggle) **Equipe:** InsurMinds — I2A2 **Data:** Julho de 2026 **Metodologia:** CRISP-DM (Cross-Industry Standard Process for Data Mining)

1. Introdução

Este relatório documenta o processo iterativo de desenvolvimento de modelos de Machine Learning para o desafio **Titanic: Machine Learning from Disaster** da plataforma Kaggle. O objetivo era construir um modelo de classificação binária capaz de prever a sobrevivência dos passageiros do Titanic, utilizando atributos como sexo, idade, classe socioeconômica e vínculos familiares.

Ao longo de **4 iterações (V1 a V4)**, experimentamos diferentes algoritmos, estratégias de engenharia de atributos (*feature engineering*) e técnicas de otimização de hiperparâmetros. Cada versão foi submetida ao Kaggle, e o resultado público foi registrado para análise comparativa.

O dataset de treino possui **891 registros** com 12 variáveis, e o dataset de teste contém **418 registros** sem a variável alvo (**Survived**). Trata-se de um dataset pequeno, o que, como veremos, teve impacto direto nas decisões de modelagem.

2. Visão Geral das Iterações

Versão	Algoritmo Principal	Nº de Features	CV Local (5-Fold)	Score Kaggle
V1	Random Forest	15	83.84%	0.78468
V2	Voting Ensemble (LR + RF + XGBoost + LightGBM)	22+	84.62%	0.76555
V3	XGBoost com GridSearchCV	17	85.52%	0.76315
V4	Random Forest (refinado)	15	83.72%	0.77511

3. Versão 1 — Random Forest (Baseline)

3.1 Abordagem

A primeira versão seguiu uma abordagem clássica e fundamentada na metodologia CRISP-DM. Após uma Análise Exploratória de Dados (EDA) completa, foram formuladas 8 hipóteses científicas (H1 a H8) sobre os fatores que influenciaram a sobrevivência. Todas foram confirmadas estatisticamente.

3.2 Engenharia de Atributos

As seguintes features foram criadas a partir dos dados brutos:

Feature	Descrição	Justificativa
FamilySize	SibSp + Parch + 1	Famílias pequenas (2-4) tiveram maior sobrevivência
IsAlone	Flag: FamilySize == 1	Passageiros sós tiveram ~30% de sobrevivência vs ~51% acompanhados
Title	Título social extraído do nome (Mr, Mrs, Miss, Master, Rare)	Títulos como Mrs/Miss superaram 70% de sobrevivência
CabinKnown	Flag: Cabin não nula	Passageiros com cabine registrada: ~67% de sobrevivência
Deck	Primeira letra da Cabin (A-G, M para missing)	Indica o convés e, indiretamente, a proximidade aos botes
TicketGroupSize	Contagem de passageiros com o mesmo bilhete	Indica grupos viajando juntos
AgeGroup	Faixas etárias fixas: Child, Teen, Adult, Senior	Crianças tiveram ~58% de sobrevivência
FareGroup	Quartis da tarifa: Low, Medium, High, VeryHigh	Tarifas altas correlacionam com sobrevivência

3.3 Modelo e Pré-processamento

- **Pré-processamento:** `ColumnTransformer` com `SimpleImputer` (mediana para numéricos, moda para categóricos), `StandardScaler` e `OneHotEncoder`.
- **Algoritmo:** `RandomForestClassifier` com `n_estimators=100`, `max_depth=6` e `min_samples_split=5`.
- **Validação:** `StratifiedKFold` com 5 folds.

3.4 Resultado

- **CV Local:** 83.84% ± 1.32%
- **Kaggle:** 0.78468

4. Versão 2 — Voting Ensemble

4.1 Motivação

Buscando superar a V1, a segunda iteração adotou uma estratégia de **Ensemble por Votação** (*Voting Classifier* com *soft voting*), combinando 4 algoritmos diferentes: Regressão Logística, Random Forest, XGBoost e LightGBM. Além disso, foram adicionadas novas features de engenharia.

4.2 Novas Features

Feature	Descrição
FarePerPerson	Tarifa dividida pelo tamanho do grupo no ticket
TicketPrefix	Prefixo alfanumérico do bilhete (top 10 + OTHER)
FamilyCategory	Categorização: Solo, Small, Medium, Large
IsMother	Flag: mulher com filhos e idade > 18
IsChild	Flag: idade < 12 anos
DeckNum	Deck como valor numérico ordinal (A=1, ..., G=7)
TitleNum	Título como encoding ordinal

4.3 Problemas Identificados

Após a submissão, o score caiu significativamente. A análise *post-mortem* revelou:

- Inconsistência no FareGroup** : A função `pd.qcut` foi aplicada separadamente ao treino e ao teste. Como os quartis são calculados com base na distribuição de cada dataset, os limites de cada faixa ficaram diferentes entre treino e teste, gerando inconsistência.
- Inconsistência no TicketPrefix** : Os 10 prefixos mais frequentes foram calculados independentemente em cada dataset, resultando em categorias diferentes.
- Diluição do Ensemble**: A Regressão Logística (modelo mais fraco, com acurácia inferior) participou da votação com peso igual, reduzindo a qualidade das previsões dos modelos mais fortes.
- Excesso de features**: Várias features novas adicionaram ruído em vez de sinal, prejudicando a capacidade de generalização.

4.4 Resultado

- **CV Local:** $84.62\% \pm 1.83\%$
 - **Kaggle:** $0.76555 \times$ (queda de 0.019 em relação à V1)
-

5. Versão 3 — XGBoost com GridSearchCV

5.1 Motivação

Para corrigir os problemas da V2, a terceira versão adotou uma abordagem de "features limpas" — removendo `FareGroup`, `TicketPrefix` e `AgeGroup` para eliminar fontes de inconsistência. O modelo escolhido foi o **XGBoost** (Extreme Gradient Boosting), com otimização de hiperparâmetros via `GridSearchCV`.

5.2 Correções Aplicadas

- Todas as transformações passaram a ser **determinísticas** (regras fixas, sem estatísticas calculadas per-dataset).
- `FarePerPerson` foi recalculado como `Fare / FamilySize` (em vez de `Fare / TicketGroupSize`).
- Modelo único (XGBoost) no lugar do Ensemble.

5.3 Otimização de Hiperparâmetros

O `GridSearchCV` testou **576 combinações** de hiperparâmetros $\times$ 5 folds = **2.880 fits no total**:

Parâmetro	Valores Testados
<code>n_estimators</code>	200, 300, 400
<code>max_depth</code>	3, 4
<code>learning_rate</code>	0.03, 0.05, 0.08
<code>subsample</code>	0.75, 0.85
<code>colsample_bytree</code>	0.75, 0.85
<code>reg_alpha</code>	0.0, 0.1
<code>reg_lambda</code>	1.0, 1.5
<code>min_child_weight</code>	1, 3

Melhores hiperparâmetros encontrados: `max_depth=3` , `learning_rate=0.08` , `n_estimators=200` , `subsample=0.75` .

5.4 Problemas Identificados

Apesar do CV mais alto de todas as versões (85.52%), o score no Kaggle foi o **pior das 4 iterações**. A causa raiz foi o **overfitting ao próprio processo de validação cruzada**:

- Com 2.880 fits em apenas 891 amostras, o GridSearchCV encontrou hiperparâmetros que **maximizaram os 5 folds específicos**, mas não generalizaram para dados novos.
- O XGBoost, por ser um algoritmo de *boosting* iterativo, é particularmente suscetível a memorizar padrões espúrios em datasets pequenos.
- Remover features categóricas como `AgeGroup` e `FareGroup` (que a V1 usava) também pode ter eliminado informação útil.

5.5 Resultado

- **CV Local:** 85.52% $\pm$ 2.11% (o MAIOR de todas as versões)
- **Kaggle:** 0.76315 $\times$ (queda de 0.022 em relação à V1 — o PIOR resultado)

6. Versão 4 — Random Forest Refinada

6.1 Motivação

Após os resultados de V2 e V3, ficou evidente que a V1 era o modelo mais robusto. A estratégia da V4 foi partir **exatamente da V1** e fazer apenas **duas melhorias cirúrgicas**, sem alterar a arquitetura nem adicionar complexidade.

6.2 Alterações (apenas 2)

Melhoria 1 — Imputação de Age pela mediana do Título:

Em vez de imputar valores ausentes de `Age` com a mediana global (~28 anos para todos), a V4 utilizou a mediana de cada grupo de título social, calculada no treino:

Título	Mediana de Idade	Perfil
Master	3.5 anos	Meninos jovens
Miss	21.0 anos	Mulheres solteiras
Mr	30.0 anos	Homens adultos
Mrs	35.0 anos	Mulheres casadas
Rare	48.5 anos	Títulos nobiliárquicos/profissionais

Melhoria 2 — Preenchimento de Embarked:

Os 2 registros com `Embarked` ausente foram preenchidos com `'S'` (Southampton, a moda) antes de entrar no pipeline, em vez de depender da imputação automática.

6.3 Resultado

- **CV Local:** 83.72% $\pm$ 1.19%
- **Kaggle:** 0.77511 (queda de 0.010 em relação à V1)
- **Diferença vs V1:** Apenas **8 passageiros** tiveram a predição alterada.

Embora a imputação por título seja uma técnica reconhecida na literatura, neste dataset específico, os 8 passageiros afetados tiveram predições ligeiramente piores do que com a imputação global da V1.

7. Ranking Final

Posição	Versão	Score Kaggle	Observação
1º	V1 — Random Forest	0.78468	Modelo mais simples e mais eficaz
2º	V4 — RF + Age por Título	0.77511	Mudança mínima, resultado próximo
3º	V2 — Voting Ensemble	0.76555	Complexidade excessiva
4º	V3 — XGBoost tuned	0.76315	Overfitting ao CV

8. Lições Aprendidas

8.1 Simplicidade vs. Complexidade

A principal lição deste projeto é que, em datasets com poucos exemplos (~891 linhas), **modelos mais simples tendem a generalizar melhor**. O Random Forest com parâmetros conservadores (`max_depth=6`, 100 árvores) superou combinações sofisticadas de XGBoost, LightGBM e Ensembles.

8.2 O Paradoxo do CV Alto

Observamos uma **correlação inversa** entre o score de validação cruzada local e o score real no Kaggle:

- V3 teve o **maior CV** (85.52%) e o **menor Kaggle** (0.76315)
- V1 teve o **menor CV** (83.84%) e o **maior Kaggle** (0.78468)

Isso acontece porque a validação cruzada em dados pequenos pode ser excessivamente otimista. Modelos com muitos hiperparâmetros ajustados acabam se especializando nos folds de validação sem realmente aprender padrões que se mantenham em dados não vistos.

8.3 GridSearchCV em Dados Pequenos

O `GridSearchCV` com muitas combinações (2.880 fits) em 891 amostras é uma receita para *overfitting ao próprio processo de validação*. O modelo encontra a combinação perfeita de hiperparâmetros para aqueles 5 folds específicos, mas não para o mundo real. Em datasets pequenos, é preferível utilizar poucos hiperparâmetros com valores conservadores baseados em experiência empírica.

8.4 Consistência nas Transformações

Funções como `pd.qcut` (que calcula quartis) devem ter seus limites definidos no treino e aplicados ao teste, nunca calculados independentemente. Na V2, essa inconsistência introduziu ruído que prejudicou as previsões.

9. Conclusão

O melhor resultado obtido foi **0.78468** na V1, colocando o modelo **acima da média geral** da competição Titanic no Kaggle. A experiência de 4 iterações demonstrou, na prática, um princípio fundamental do Machine Learning: **a complexidade do modelo deve ser proporcional ao volume de dados disponível**.

Para datasets pequenos como o Titanic, a combinação de uma boa Análise Exploratória (EDA), engenharia de atributos fundamentada em hipóteses de negócio e um algoritmo robusto como o Random Forest com parâmetros conservadores continua sendo a abordagem mais confiável.

Relatório elaborado como parte do Desafio 03 — InsurMinds (I2A2)